

Saving Money on Long Coding-Agent Sessions

Project report – analysis of post-session coding agent traces (SWE-chat dataset)

1 What I did

I worked with **SWE-chat**, a public dataset from Stanford with about 6,000 real coding-agent sessions from developers using tools like Claude Code. My goal was to estimate how much money could be saved by splitting up long sessions, and to fill the empty table cell “upper bound of saved money”.

- I focused on **long sessions** (more than 30 turns), because these are where the context grows large and costs go up.
- I built a **cost model** using the real token counts in the data and Claude’s official prices (Opus: \$5 per million input tokens, \$25 output; cache reads are 90% cheaper).
- I broke the cost into **four token types** (fresh input, output, cache writes, cache reads) to see which part is actually expensive.
- I opened the raw turn-by-turn transcripts and ran a **split simulation**: every time the context passed a limit, I “split” the session into a new part that does not carry the old history, and recomputed the cost.

2 What I saw

Long sessions are a small part of all sessions, but they cost a very large part of the money:

Finding	Result
Total spend (all sessions)	\$51,204
Share of sessions that are long (>30 turns)	14%
Share of total money spent on long sessions	49% (\$25,076)
Cache reads (re-reading old context)	63% of long-session cost
Output tokens (cannot be reduced by splitting)	6% of long-session cost

The most important thing I saw: most of the cost (**63%**) is cache reads – the old conversation history being re-read on every turn. But cache reads are already discounted by 90%, so the “expensive-looking” big context is actually the cheap part.

When I simulated splitting the long sessions (on a sample of about 300 real sessions), the savings were:

Splitting assumption	Saving
Free handoff (perfect, impossible – the ceiling)	~30%
Real handoff (with a summary cost per split)	~16%

Free vs. real handoff: when you split a session, the new part does not know what happened before. In real life you must give it a short summary, and that summary costs tokens. “Free handoff” pretends the summary is free (best case). “Real handoff” charges a realistic cost. The gap between 30% and 16% is the price of these summaries.

3 What I concluded

The simple guess – “long sessions waste a lot of money on big context, so splitting saves a fortune” – is not correct. The big context is mostly cache reads, which are already cheap. So the real saving is much smaller than the \$25,076 baseline. Splitting also is not free: each split needs a summary, and the sessions are already doing some automatic compaction by themselves.

My estimate for the saving: about \$4,000 realistic, with an upper bound (best case) of about \$7,500.

How \$4,000 and \$7,500 were calculated: the simulation gives savings as a percentage (16% realistic, 30% best case). These percentages are applied to the total cost of all long sessions, **\$25,076**: $16\% \times \$25,076 \approx \$4,000$, and $30\% \times \$25,076 \approx \$7,500$. They are scaled estimates from the sample, not directly measured on every session.

4 What to put in the table cell

For the empty cell “upper bound of saved money”, I suggest writing a range with a short reason, not a single number:

Upper bound $\approx$ \$7,500 (best case, 30% of long-session cost, free/lossless splits).

Realistic estimate $\approx$ \$4,000 (16%, with handoff summary cost).

Method: simulated context-limit splits over $\sim$ 300 real Claude Code transcripts, scaled to all 797 long sessions (\$25,076 total). The figure is far below the baseline because 63% of the cost is cache reads (already 90% discounted) and auto-compaction is already partly active.